Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

SDN & Network Softwarization — TP6 — Microsegmentation et ACL distribuées sur trois switches OVS
4e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. BOUHORMA

Date : 6 novembre 2026

Table des matières

Cadre du TP **2**

1 Créer et comprendre la topologie **4**
1.1 Définir la topologie 4
1.2 Lancer Mininet 5
1.3 Observer la topologie 5
1.4 Fixer le mode `secure` et vérifier les ports 6
1.5 Wireshark 7

2 Observer un switch sans politique **7**

3 Rendre le réseau communicant : ARP puis IPv4 **9**
3.1 Autoriser la résolution ARP sur les trois switches 9
3.2 Programmer le forwarding IPv4 par destination 10
3.3 Tester la communication de référence 12
3.4 Vérifier le transit dans `s2` 13

4 ACL ICMP sur `s1` : bloquer le ping initié par `h2` **14**

5 ACL TCP sur `s3` : protéger le serveur HTTP contre `h4` **16**

6 Valider la politique et interpréter les résultats **18**
6.1 Questions de compréhension 20

7 Clôture du TP **20**

8 Synthèse **21**

Cadre du TP

Plateforme	**Outils**	**Mode**
Ubuntu (portable `Taha-inspiron-16`)	Mininet, Open vSwitch (`ovs-vsctl`, `ovs-ofctl`), OpenFlow 1.3, Wireshark graphique	Individuel
Approche	**Switchs / contrôleur**	**Livrable**
Forwarding ARP et IPv4 installé à la main sur trois switches, puis deux ACL ciblées (ICMP sur `s1`, TCP sur `s3`) vérifiées dans Wireshark et avec `n_packets`	Trois `OVSSwitch` (`ovsk`) en OpenFlow 1.3, `fail-mode secure`, aucun contrôleur	Compte rendu avec captures

TABLE 1 – Cadre général du TP

Question de départ : *peut-on interdire un protocole à un utilisateur tout en conservant les autres services, et en plaçant les ACL à différents endroits du réseau ?*

Objectifs :

— lire une topologie multi-switches et associer les numéros de ports OpenFlow aux liens ;

— expliquer pourquoi ARP et le forwarding IPv4 doivent être programmés avant toute ACL ;

— traduire deux politiques de sécurité en règles OpenFlow (source, destination, protocole, port et priorité) ;

— vérifier l'effet d'un `DROP` dans Wireshark et dans les compteurs `n_packets` ;

— justifier le choix du switch sur lequel chaque règle ACL est installée.

Scénario. `h1` est l'administrateur (10.0.0.1), `h2` l'utilisateur (10.0.0.2), `h3` le serveur HTTP (10.0.0.3, TCP 8080) et `h4` l'invité (10.0.0.4). Les quatre hôtes sont dans 10.0.0.0/24, répartis sur trois switches en chaîne : `s1` — `s2` — `s3`.

Politique de sécurité à obtenir :

Flux à tester	Avant ACL	Après ACL
`h1` → `h3` : ping / HTTP 8080	Autorisé	Autorisé
`h2` → `h3` : ping	Autorisé	Interdit
`h2` → `h3` : HTTP 8080	Autorisé	Autorisé
`h4` → `h3` : HTTP 8080	Autorisé	Interdit
`h4` → `h3` : ping	Autorisé	Autorisé

TABLE 2 – Politique de sécurité visée

Méthode constante :

1. Terminal 2 : installer la règle (`ovs-ofctl`) ;
2. Wireshark : *Capture* → *Redémarrer*, puis appliquer le filtre d'affichage ;
3. Terminal 1 (`mininet>`) : générer le trafic ;
4. Wireshark : comparer l'interface d'entrée et l'interface de sortie ;
5. Terminal 2 : lire `n_packets` avec `dump-flows`.

Note sur l'environnement. Manipulations faites sur mon portable Ubuntu, dans le dossier `~/Bureau/UIR/4A/S7/SDN/TP6`, avec trois fenêtres : le Terminal 1 pour Mininet, le Terminal 2 pour `ovs-vsctl` et `ovs-ofctl`, et Wireshark en mode graphique, lancé sans `sudo` avec un filtre de capture vide. Toutes les commandes `ovs-ofctl` portent l'option `-O OpenFlow13`.

1 Créer et comprendre la topologie

Objectif : connaître la position des hôtes et les numéros de ports OpenFlow *avant* de programmer les actions `output:N`, sinon une règle correcte sur le papier peut transmettre vers le mauvais lien.

1.1 Définir la topologie

Listing 1 – Terminal 2 : création du fichier de topologie

```
nano ~/topo_microseg.py
```

Listing 2 – Fichier `topo_microseg.py`

```
from mininet.topo import Topo

class MicrosegTopo(Topo):
    def build(self):
        s1 = self.addSwitch('s1')
        s2 = self.addSwitch('s2')
        s3 = self.addSwitch('s3')
        h1 = self.addHost('h1')
        h2 = self.addHost('h2')
        h3 = self.addHost('h3')
        h4 = self.addHost('h4')

        self.addLink(h1, s1)   # s1-eth1
        self.addLink(h2, s1)   # s1-eth2
        self.addLink(s1, s2)   # s1-eth3 / s2-eth1
        self.addLink(s2, s3)   # s2-eth2 / s3-eth1
        self.addLink(h3, s3)   # s3-eth2
        self.addLink(h4, s3)   # s3-eth3

topos = {'microseg': lambda: MicrosegTopo()}
```

Ce fichier décrit les équipements et les liens : il n'installe aucune ACL. L'ordre de création des liens fixe les ports attendus.

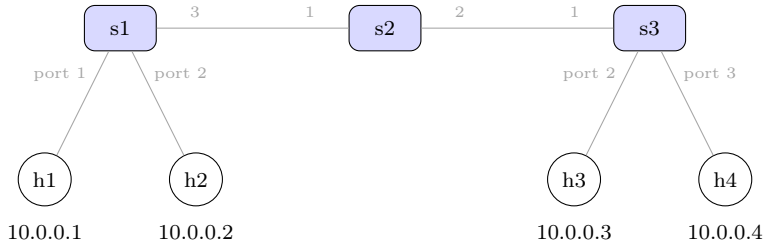


FIGURE 1 – Topologie du TP : trois switches en chaîne

Switch	Port 1	Port 2	Port 3
s1	h1	h2	vers s2
s2	vers s1	vers s3	—
s3	vers s2	h3 (HTTP)	h4 (invité)

TABLE 3 – Ports OpenFlow attendus

1.2 Lancer Mininet

Listing 3 – Terminal 1 : nettoyage puis création du réseau

```
sudo mn -c
sudo mn --custom ~/topo_microseg.py --topo microseg \
  --switch ovsk,protocols=OpenFlow13 --controller none
```

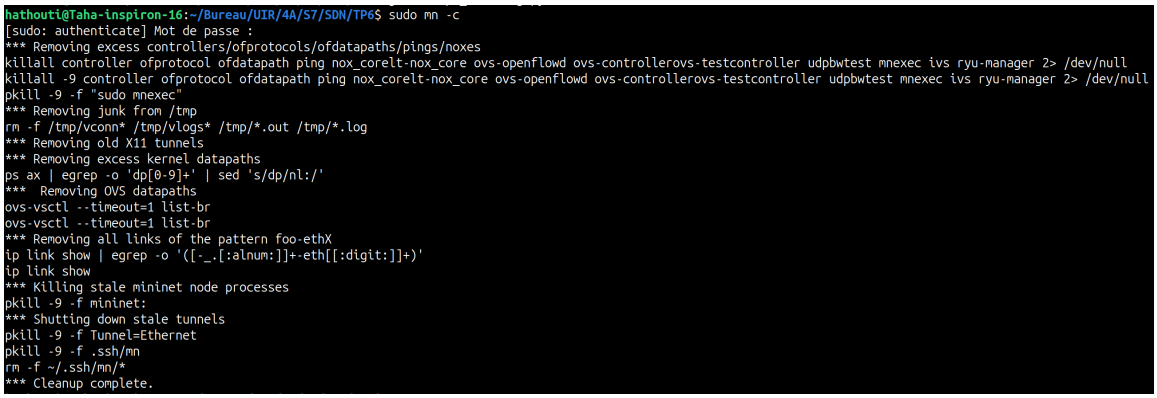


FIGURE 2 – Nettoyage initial avec `sudo mn -c`



FIGURE 3 – Création de la topologie `microseg` sans contrôleur

1.3 Observer la topologie

Listing 4 – Terminal 1 : liens et équipements

```
mininet> net
mininet> nodes
```

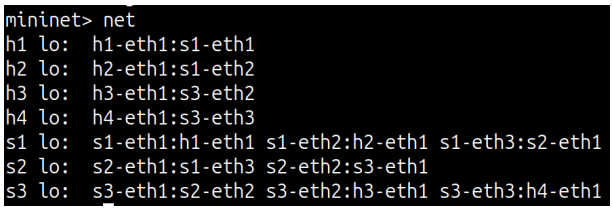


FIGURE 4 – Sortie de `net`

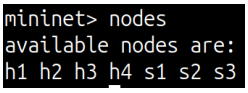


FIGURE 5 – Sortie de `nodes`

Observation. `net` donne `h1-eth1:s1-eth1`, `h2-eth1:s1-eth2`, `h3-eth1:s3-eth2` et `h4-eth1:s3-eth3`, puis les liaisons inter-switches `s1-eth3:s2-eth1` et `s2-eth2:s3-eth1`. `nodes` liste `h1`, `h2`, `h3`, `h4`, `s1`, `s2` et `s3` : trois switches et quatre hôtes, comme attendu.

1.4 Fixer le mode `secure` et vérifier les ports

Listing 5 – Terminal 2 : mode de panne puis ports des trois switches

```
sudo ovs-vsctl set-fail-mode s1 secure
sudo ovs-vsctl set-fail-mode s2 secure
sudo ovs-vsctl set-fail-mode s3 secure
sudo ovs-ofctl -O OpenFlow13 show s1
sudo ovs-ofctl -O OpenFlow13 show s2
sudo ovs-ofctl -O OpenFlow13 show s3
```

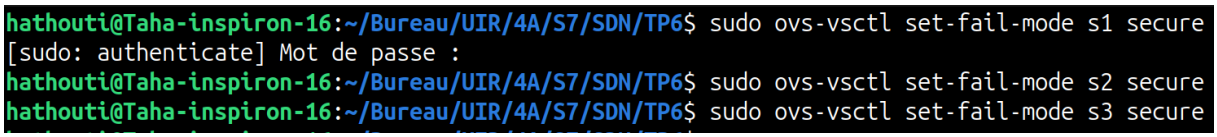


FIGURE 6 – `set-fail-mode secure` sur `s1`, `s2` et `s3`

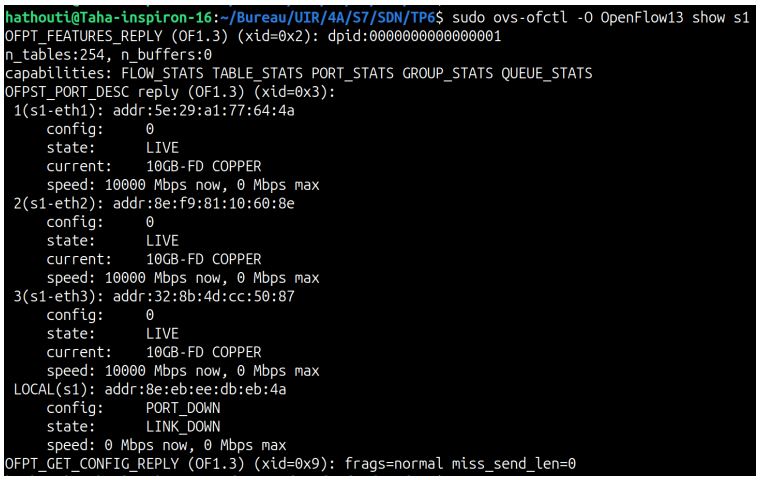


FIGURE 7 – Sortie de `ovs-ofctl -O OpenFlow13 show s1`

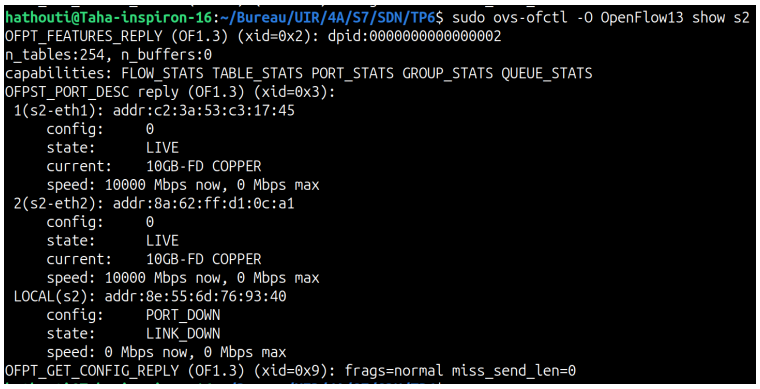


FIGURE 8 – Sortie de `ovs-ofctl -O OpenFlow13 show s2`

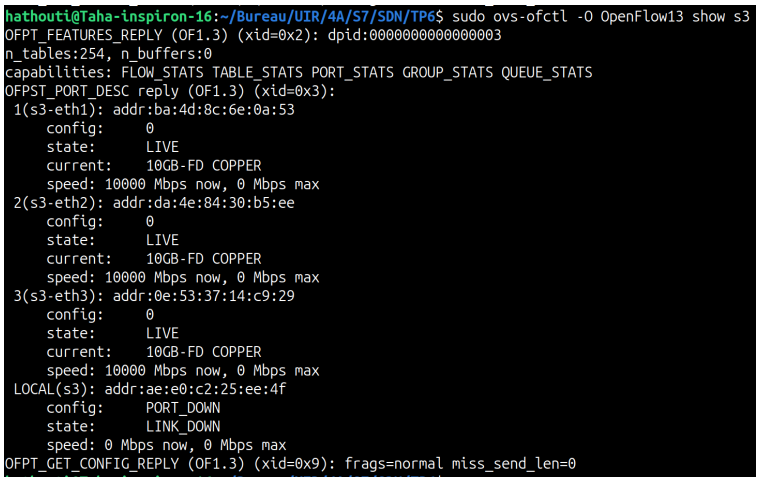


FIGURE 9 – Sortie de `ovs-ofctl -O OpenFlow13 show s3`

Observation. Chaque `show` répond en OpenFlow 1.3 (`n_tables:254`) et liste ses ports à l'état `LIVE` (10 Gbit/s), plus le port interne `LOCAL` (`PORT_DOWN`, hors TP). Pour `s1`, les ports 1, 2 et 3 sont `s1-eth1`, `s1-eth2` et `s1-eth3`, conformément au tableau des ports attendus. La ligne `miss_send_len=0` indique qu'aucun paquet n'est envoyé à un contrôleur en l'absence de règle.

Checkpoint 1. Les ports réels correspondent au tableau : les actions `output:N` de la suite n'ont pas besoin d'être adaptées. Un `output:3` ne désigne pas une adresse IP mais le port 3 du switch concerné.

1.5 Wireshark

Wireshark est lancé depuis un terminal avec `wireshark &`, sans `sudo`. Dans *Capture* → *Options*, on coche les deux interfaces utiles à chaque étape (entrée et sortie du switch observé), le filtre de capture reste vide et les filtres sont saisis dans la barre d'affichage. Avant chaque test, *Capture* → *Redémarrer* vide les paquets précédents.

2 Observer un switch sans politique

Objectif : démontrer que la présence des câbles virtuels et des adresses IP ne garantit pas la communication : sans règle applicable, les trames sont abandonnées.

Listing 6 – Terminal 2 : tables de flux vides

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
sudo ovs-ofctl -O OpenFlow13 del-flows s2
sudo ovs-ofctl -O OpenFlow13 del-flows s3
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

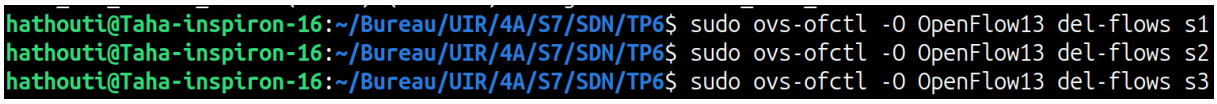


FIGURE 10 – Suppression des règles de `s1`, `s2` et `s3`

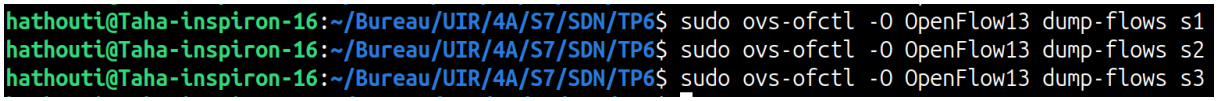


FIGURE 11 – Vérification : tables de flux vides

Listing 7 – Terminal 1 : trafic (Wireshark sur `s1-eth1` et `s1-eth3`, filtre : `arp || icmp`)

```
mininet> h1 ip neigh flush all
mininet> h3 ip neigh flush all
mininet> h1 ping -c 2 h3
```

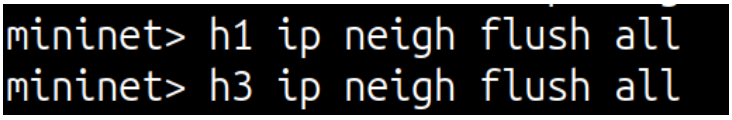


FIGURE 12 – Vidage des caches ARP de `h1` et `h3`

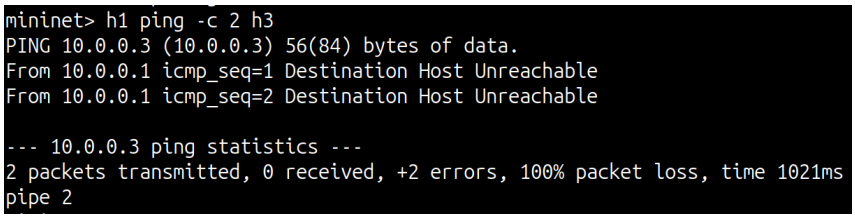


FIGURE 13 – Sortie de `h1 ping -c 2 h3`

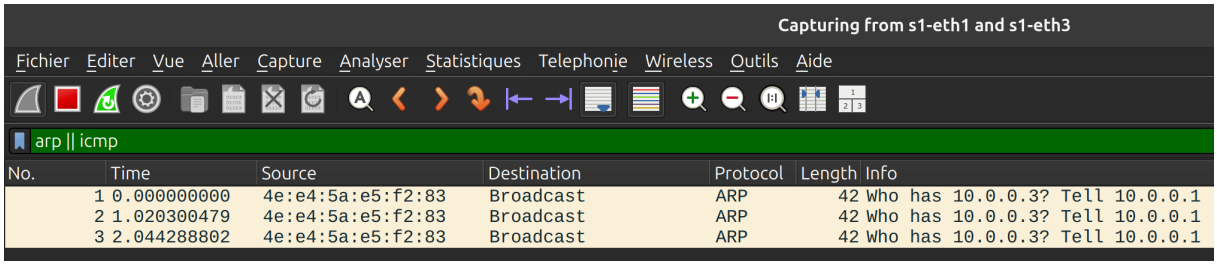


FIGURE 14 – Capture Wireshark (filtre `arp || icmp`) sur `s1-eth1` et `s1-eth3`

Observation. Le ping échoue : 2 émis, 0 reçu, `+2 errors`, 100 % de perte, avec `From 10.0.0.1 icmp_seq=1 Destination Host Unreachable`. Dans Wireshark, seules des ARP Request `Who has 10.0.0.3? Tell 10.0.0.1` apparaissent (à 0, 1 et 2 secondes), sans aucune ARP Reply ni aucun ICMP.

Question : quelle différence existe entre « le lien est UP » et « le switch sait transmettre les trames » ?

Un lien `UP` signifie seulement que l'interface est active : la trame arrive bien sur le switch (elle est capturée sur `s1-eth1`). Pour qu'elle ressorte, il faut en plus une règle qui indique l'action à appliquer. En mode `secure` et sans règle, `s1` ne retransmet rien, `h1` ne résout pas l'adresse MAC de `h3` et son noyau déclare l'échec. Un ping en échec n'est donc pas forcément un problème physique.

3 Rendre le réseau communicant : ARP puis IPv4

Objectif : une ACL n'a de sens que si la communication fonctionne *avant* son installation. On programme d'abord la résolution MAC, puis le chemin aller et retour des paquets IPv4.

3.1 Autoriser la résolution ARP sur les trois switches

Listing 8 – Terminal 2 : règles ARP

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,arp,actions=FLOOD"
sudo ovs-ofctl -O OpenFlow13 add-flow s2 "priority=100,arp,actions=FLOOD"
sudo ovs-ofctl -O OpenFlow13 add-flow s3 "priority=100,arp,actions=FLOOD"
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,arp,actions=FLOOD"
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 add-flow s2 "priority=100,arp,actions=FLOOD"
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 add-flow s3 "priority=100,arp,actions=FLOOD"
```

FIGURE 15 – Ajout de la règle `arp,actions=FLOOD` sur les trois switches

Listing 9 – Terminal 1 : ARP seul (filtre Wireshark : `arp || icmp`)

```
mininet> h1 ip neigh flush all
mininet> h3 ip neigh flush all
mininet> h1 ping -c 1 h3
mininet> h1 ip neigh show
```

```
mininet> h1 ip neigh flush all
mininet> h3 ip neigh flush all
```

FIGURE 16 – Vidage des caches ARP de `h1` et `h3`

```
mininet> h1 ping -c 1 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.

--- 10.0.0.3 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms
```

FIGURE 17 – Sortie de `h1 ping -c 1 h3`

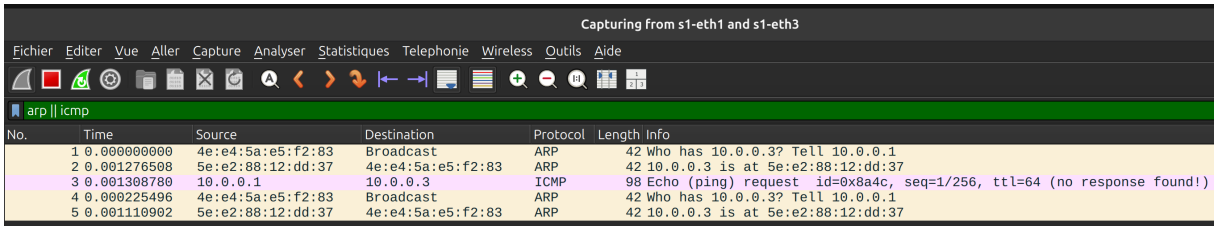


FIGURE 18 – Capture Wireshark : ARP résolu, Echo Request sans réponse

```
mininet> h1 ip neigh show
10.0.0.3 dev h1-eth0 lladdr 5e:e2:88:12:dd:37 STALE
```

FIGURE 19 – Sortie de `h1 ip neigh show`

Observation. Le ping échoue : 1 émis, 0 reçu, 100 % de perte, cette fois **sans** `Destination Host Unreachable`. Dans Wireshark, l'ARP Request et l'ARP Reply (`10.0.0.3 is at 5e:e2:88:12:dd:37`) sont visibles, et l'Echo Request n'apparaît qu'une seule fois, avec la mention `no response found`. `ip neigh show` confirme que `h1` connaît désormais l'adresse MAC de `h3` (état `STALE`).

Interprétation. `h1` apprend l'adresse MAC de `h3` et envoie réellement son paquet ICMP, mais celui-ci ne correspond à aucune règle et est abandonné : **l'ARP résout une adresse MAC, il ne remplace pas les règles IPv4**.

3.2 Programmer le forwarding IPv4 par destination

Chaque switch choisit le port de sortie d'après l'adresse IP de destination. Pour rejoindre `h3` depuis `h1` : `s1` envoie vers `s2` (port 3), `s2` vers `s3` (port 2), `s3` vers `h3` (port 2). Le retour utilise les sorties inverses. Il faut donc $4 \times 3 = 12$ règles, car chaque switch doit savoir traiter les quatre destinations.

Listing 10 – Terminal 2 : règles IPv4 de `s1`

```
1 sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst
    =10.0.0.1,actions=output:1"
2 sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst
    =10.0.0.2,actions=output:2"
3 sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst
    =10.0.0.3,actions=output:3"
4 sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst
    =10.0.0.4,actions=output:3"
5 sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst=10.0.0.1,actions=output:1"
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst=10.0.0.2,actions=output:2"
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst=10.0.0.3,actions=output:3"
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst=10.0.0.4,actions=output:3"
```

FIGURE 20 – Règles IPv4 de `s1`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=15.488s, table=0, n_packets=0, n_bytes=0, priority=200,ip,nw_dst=10.0.0.1 actions=output:"s1-eth1"
 cookie=0x0, duration=10.519s, table=0, n_packets=0, n_bytes=0, priority=200,ip,nw_dst=10.0.0.2 actions=output:"s1-eth2"
 cookie=0x0, duration=5.870s, table=0, n_packets=0, n_bytes=0, priority=200,ip,nw_dst=10.0.0.3 actions=output:"s1-eth3"
 cookie=0x0, duration=2.656s, table=0, n_packets=0, n_bytes=0, priority=200,ip,nw_dst=10.0.0.4 actions=output:"s1-eth3"
 cookie=0x0, duration=576.789s, table=0, n_packets=2, n_bytes=84, priority=100,arp actions=FLOOD
```

FIGURE 21 – Table de flux de `s1` après l'ajout

Listing 11 – Terminal 2 : règles IPv4 de `s2`

```
sudo ovs-ofctl -O OpenFlow13 add-flow s2 "priority=200,ip,nw_dst
   =10.0.0.1,actions=output:1"
sudo ovs-ofctl -O OpenFlow13 add-flow s2 "priority=200,ip,nw_dst
   =10.0.0.2,actions=output:1"
sudo ovs-ofctl -O OpenFlow13 add-flow s2 "priority=200,ip,nw_dst
   =10.0.0.3,actions=output:2"
sudo ovs-ofctl -O OpenFlow13 add-flow s2 "priority=200,ip,nw_dst
   =10.0.0.4,actions=output:2"
sudo ovs-ofctl -O OpenFlow13 dump-flows s2
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 add-flow s2 "priority=200,ip,nw_dst=10.0.0.1,actions=output:1"
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 add-flow s2 "priority=200,ip,nw_dst=10.0.0.2,actions=output:1"
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 add-flow s2 "priority=200,ip,nw_dst=10.0.0.3,actions=output:2"
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 add-flow s2 "priority=200,ip,nw_dst=10.0.0.4,actions=output:2"
```

FIGURE 22 – Règles IPv4 de `s2`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 dump-flows s2
 cookie=0x0, duration=18.086s, table=0, n_packets=0, n_bytes=0, priority=200,ip,nw_dst=10.0.0.1 actions=output:"s2-eth1"
 cookie=0x0, duration=14.427s, table=0, n_packets=0, n_bytes=0, priority=200,ip,nw_dst=10.0.0.2 actions=output:"s2-eth1"
 cookie=0x0, duration=8.578s, table=0, n_packets=0, n_bytes=0, priority=200,ip,nw_dst=10.0.0.3 actions=output:"s2-eth2"
 cookie=0x0, duration=5.424s, table=0, n_packets=0, n_bytes=0, priority=200,ip,nw_dst=10.0.0.4 actions=output:"s2-eth2"
 cookie=0x0, duration=658.809s, table=0, n_packets=2, n_bytes=84, priority=100,arp actions=FLOOD
```

FIGURE 23 – Table de flux de `s2` après l'ajout

Listing 12 – Terminal 2 : règles IPv4 de `s3`

```
sudo ovs-ofctl -O OpenFlow13 add-flow s3 "priority=200,ip,nw_dst
   =10.0.0.1,actions=output:1"
sudo ovs-ofctl -O OpenFlow13 add-flow s3 "priority=200,ip,nw_dst
   =10.0.0.2,actions=output:1"
sudo ovs-ofctl -O OpenFlow13 add-flow s3 "priority=200,ip,nw_dst
   =10.0.0.3,actions=output:2"
sudo ovs-ofctl -O OpenFlow13 add-flow s3 "priority=200,ip,nw_dst
   =10.0.0.4,actions=output:3"
sudo ovs-ofctl -O OpenFlow13 dump-flows s3
```

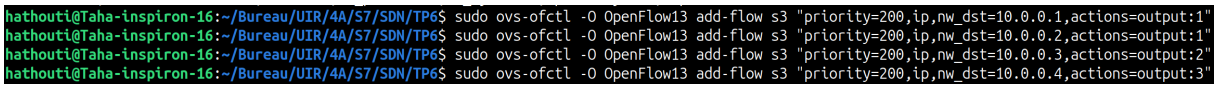


FIGURE 24 – Règles IPv4 de `s3`

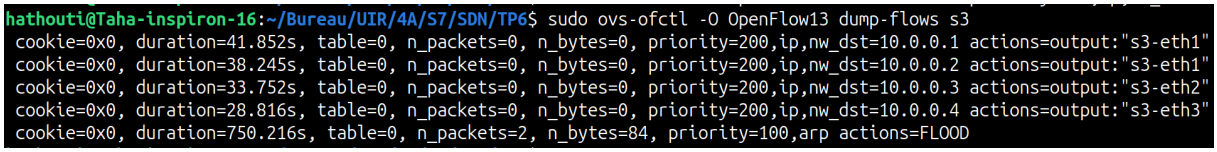


FIGURE 25 – Table de flux de `s3` après l'ajout

Observation. Chaque switch contient les quatre règles `priority=200` et la règle ARP `priority=100`. Dans `s1`, les compteurs IPv4 sont encore à `n_packets=0` (aucun paquet IP n'a été émis depuis l'installation) alors que la règle ARP affiche `n_packets=2`, `n_bytes=84`.

3.3 Tester la communication de référence

Listing 13 – Terminal 1 : connectivité et serveur HTTP

```
mininet> pingall
mininet> h3 python3 -m http.server 8080 >/tmp/microseg_http.log
   2>&1 &
mininet> h1 curl -I --max-time 3 http://10.0.0.3:8080
mininet> h2 curl -I --max-time 3 http://10.0.0.3:8080
mininet> h4 curl -I --max-time 3 http://10.0.0.3:8080
```

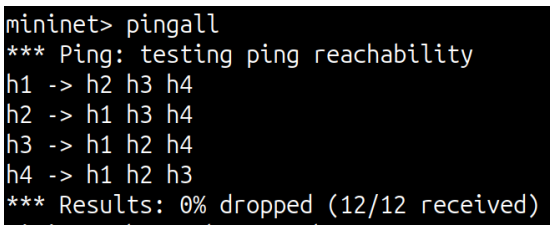


FIGURE 26 – Sortie de `pingall`

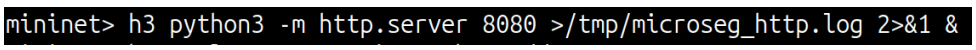


FIGURE 27 – Démarrage du serveur HTTP dans `h3` (port 8080)

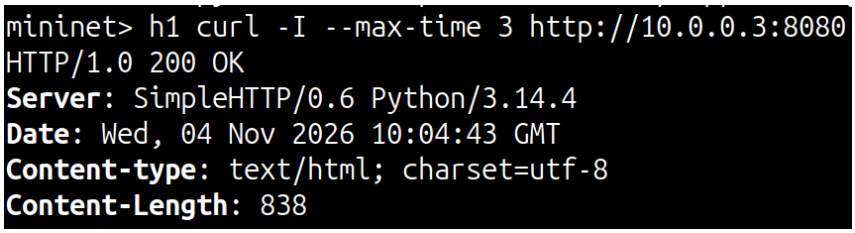


FIGURE 28 – Sortie de `h1 curl -I http://10.0.0.3:8080`

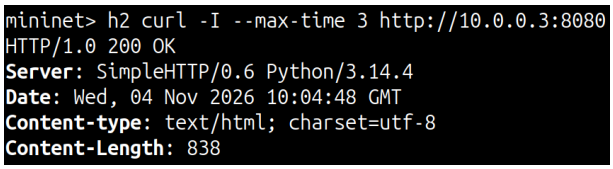


FIGURE 29 – Sortie de `h2 curl -I http://10.0.0.3:8080`

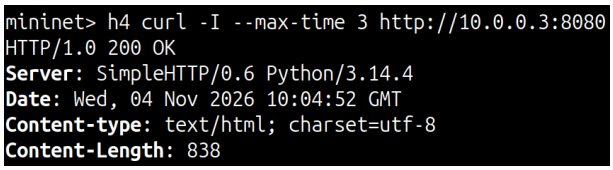


FIGURE 30 – Sortie de `h4 curl -I http://10.0.0.3:8080`

Observation. `pingall` donne `0% dropped (12/12 received)`. Les trois `curl` reçoivent `HTTP/1.0 200 OK` (serveur `SimpleHTTP/0.6 Python/3.14.4`, `Content-Length: 838`) : avant toute ACL, `h1`, `h2` et `h4` atteignent le serveur.

3.4 Vérifier le transit dans s2

Wireshark capture maintenant `s2-eth1` et `s2-eth2` (filtre : `icmp || tcp.port==8080`), afin de voir le trafic entrer dans le switch de transit et en ressortir.

Listing 14 – Terminal 1 : ping puis requête HTTP depuis `h1`

```
mininet> h1 ping -c 3 h3
mininet> h1 curl http://10.0.0.3
mininet> h1 curl http://10.0.0.3:8080
```

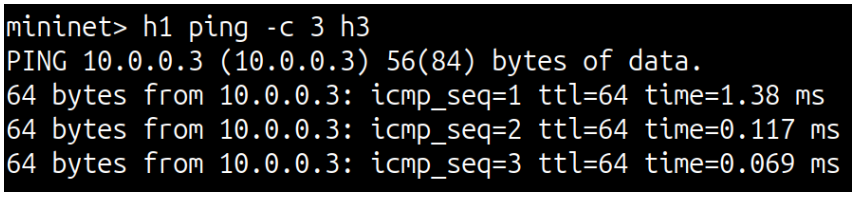


FIGURE 31 – Sortie de `h1 ping -c 3 h3`

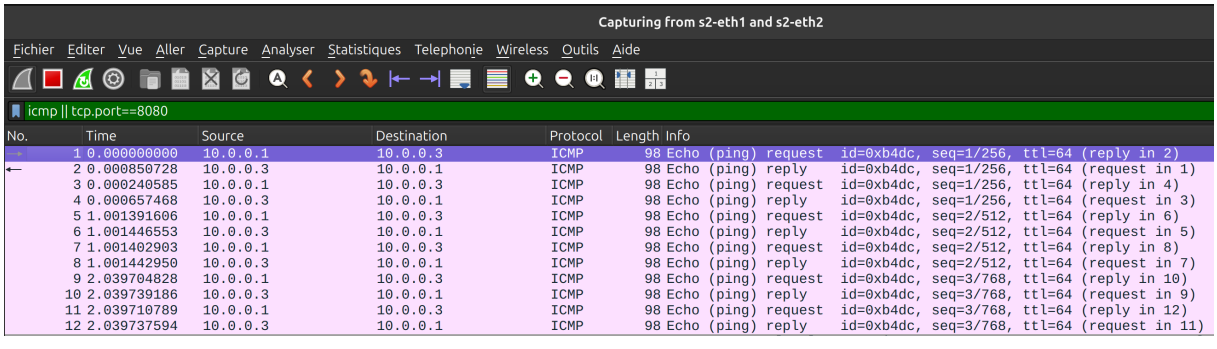


FIGURE 32 – Capture Wireshark sur `s2-eth1` et `s2-eth2` : ping `h1` → `h3`

```
mininet> h1 curl http://10.0.0.3
curl: (7) Failed to connect to 10.0.0.3 port 80 after 0 ms: Could not connect to server
mininet> h1 curl http://10.0.0.3:8080
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<style type="text/css">
:root {
color-scheme: light dark;
}
</style>
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href=".~lock.TP_OpenFlow_Microsegmentation_ACL2.docx%23">.~lock.TP_OpenFlow_Microsegmentation_ACL2.docx#</a></li>
<li><a href="2_Observer_un_switch_sans_politique.pcapng">2_Observer_un_switch_sans_politique.pcapng</a></li>
<li><a href="3_1_Autoriser_la_r%C3%A9solution_ARP_sur_les_trois_switches.pcapng">3_1_Autoriser_la_résolution_ARP_sur_les_trois_switches.pcapng</a></li>
<li><a href="Screen/">Screen/</a></li>
<li><a href="topo_microseg.py">topo_microseg.py</a></li>
<li><a href="TP_OpenFlow_Microsegmentation_ACL2.docx">TP_OpenFlow_Microsegmentation_ACL2.docx</a></li>
</ul>
<hr>
</body>
</html>
```

FIGURE 33 – Sortie de `curl` : port 80 refusé, port 8080 servi

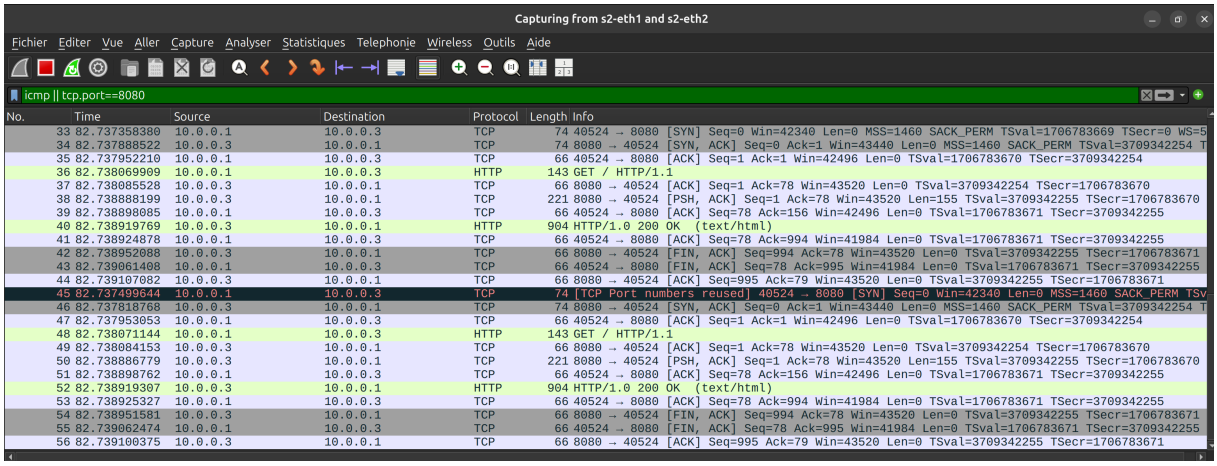


FIGURE 34 – Capture Wireshark sur `s2` : échange TCP/HTTP `h1` → `h3`

Observation. Le ping `h1` → `h3` réussit (3 Echo Request, 3 Echo Reply). Chaque paquet apparaît **deux fois** dans Wireshark : une fois à l'entrée de `s2`, une fois à sa sortie, ce qui prouve que le switch de transit relaie effectivement le trafic. Le `curl` sans port échoue (`Failed to connect to 10.0.0.3 port 80`) car le serveur écoute sur 8080 ; avec `:8080`, la page `Directory listing for /` est renvoyée. La capture montre le SYN, le SYN/ACK et l'ACK, la requête `GET / HTTP/1.1`, la réponse `HTTP/1.0 200 OK (text/html)` puis la fermeture par `FIN, ACK`, là aussi à l'entrée et à la sortie.

Checkpoint 3. Avant toute ACL, `pingall` ne perd aucun paquet et HTTP répond `200 OK` : le forwarding est validé et on peut installer les ACL.

4 ACL ICMP sur `s1` : bloquer le ping initié par `h2`

Objectif : interdire à l'utilisateur `h2` de lancer un ping vers `h3`, tout en lui laissant HTTP. L'ACL est placée sur `s1`, au plus près de `h2`.

Listing 15 – Terminal 2 : ACL ICMP précise

```
1 sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
```

```
  "priority=300,icmp,nw_src=10.0.0.2,nw_dst=10.0.0.3,icmp_type=8,
     actions=drop"
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

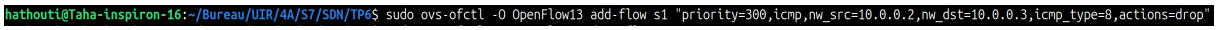


FIGURE 35 – Ajout de la règle `priority=300` / `drop` sur `s1`

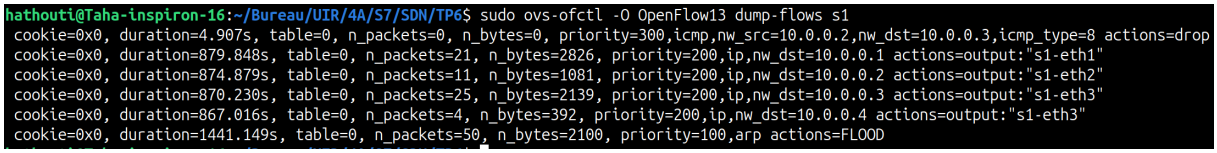


FIGURE 36 – Table de flux de `s1` avec l'ACL ICMP

Fragment	Interprétation
`priority=300`	Passe avant le forwarding IPv4 de priorité 200 si les deux règles correspondent
`icmp`	Ne concerne que les paquets ICMP
`nw_src=10.0.0.2`	Uniquement les paquets provenant de `h2`
`nw_dst=10.0.0.3`	Uniquement ceux destinés à `h3`
`icmp_type=8`	Uniquement les Echo Request (début d'un ping), pas les Echo Reply
`actions=drop`	Le paquet n'est pas transmis ; le compteur de la règle augmente

TABLE 4 – Lecture de l'ACL ICMP

Listing 16 – Terminal 1 : tests après ACL (Wireshark sur `s1-eth2` et `s1-eth3`, filtre : `icmp && ip.src==10.0.0.2 && ip.dst==10.0.0.3`)

```
mininet> h2 ping -c 3 h3
mininet> h1 ping -c 2 h3
mininet> h2 curl -I --max-time 3 http://10.0.0.3:8080
```

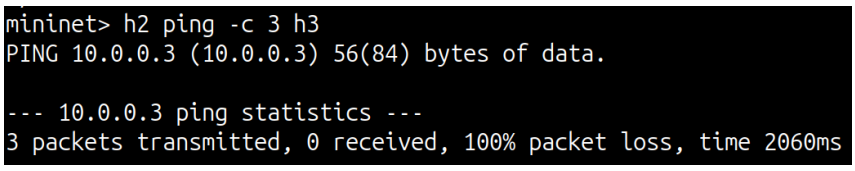


FIGURE 37 – Sortie de `h2 ping -c 3 h3`

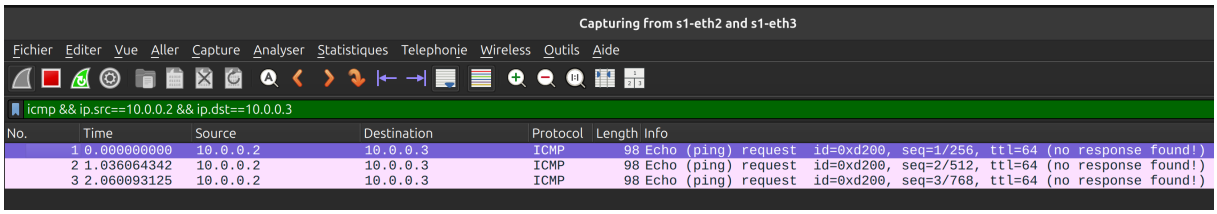


FIGURE 38 – Capture Wireshark : Echo Request `h2` → `h3` sur `s1-eth2` uniquement

```
mininet> h1 ping -c 2 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=1.44 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.112 ms

--- 10.0.0.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.112/0.777/1.443/0.665 ms
```

FIGURE 39 – Sortie de `h1 ping -c 2 h3`

```
mininet> h2 curl -I --max-time 3 http://10.0.0.3:8080
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.14.4
Date: Wed, 04 Nov 2026 10:19:13 GMT
Content-type: text/html; charset=utf-8
Content-Length: 975
```

FIGURE 40 – Sortie de `h2 curl -I http://10.0.0.3:8080`

Observation.

- `h2` → `h3` (ping) : 3 émis, 0 reçu, 100 % de perte (2060 ms). Wireshark montre trois Echo Request (`seq=1, 2, 3`) sur `s1-eth2` et aucun sur `s1-eth3` : ils sont abandonnés par `s1` ;
- `h1` → `h3` (ping) : 2/2 reçus, RTT min/moyen/max = 0,112/0,777/1,443 ms ;
- `h2` → `h3` (HTTP) : `HTTP/1.0 200 OK`.

L'ACL est donc **sélective** : seul le ping de `h2` est bloqué, la source `h1` et le protocole HTTP restent autorisés. Les compteurs de `s1` sont relus dans la partie 6.

Question : si `h3` initie un ping vers `h2`, pourquoi la règle `icmp_type=8` en sens `h2` → `h3` ne bloque-t-elle pas nécessairement la réponse ?

L'Echo Request de `h3` a pour source 10.0.0.3, il ne correspond donc pas à `nw_src=10.0.0.2`. La réponse de `h2` part bien de 10.0.0.2 vers 10.0.0.3, mais c'est un Echo Reply (type 0) et non un type 8 : elle ne correspond pas non plus à la règle et suit le forwarding IPv4 de priorité 200.

5 ACL TCP sur `s3` : protéger le serveur HTTP contre `h4`

Objectif : interdire à l'invité `h4` l'accès au service HTTP de `h3` (port 8080) tout en lui laissant le ping. Comme `h4` et `h3` sont tous deux reliés à `s3`, bloquer sur `s1` ne servirait

à rien : ce trafic ne passe jamais par `s1`.

Listing 17 – Terminal 2 : ACL TCP sur `s3`

```
sudo ovs-ofctl -O OpenFlow13 add-flow s3 \
  "priority=350,tcp,nw_src=10.0.0.4,nw_dst=10.0.0.3,tcp_dst=8080,
     actions=drop"
sudo ovs-ofctl -O OpenFlow13 dump-flows s3
```

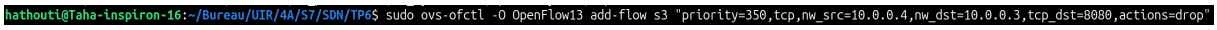


FIGURE 41 – Ajout de la règle `priority=350` / `drop` sur `s3`

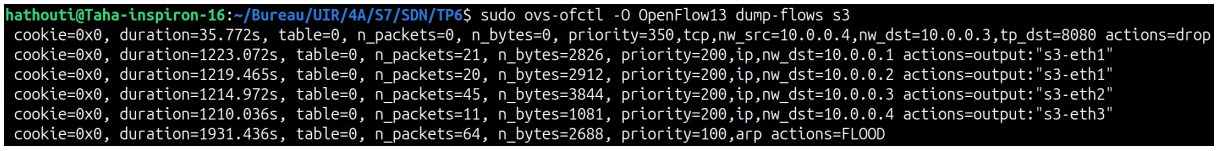


FIGURE 42 – Table de flux de `s3` avec l'ACL TCP

Observation. `tcp` cible les segments TCP, `tcp_dst=8080` vise le port du serveur (affiché `tp_dst=8080` par `dump-flows`), `nw_src` et `nw_dst` limitent le blocage à `h4` → `h3`, `priority=350` l'emporte sur les règles IPv4 de priorité 200, et `drop` supprime notamment le SYN de connexion.

Listing 18 – Terminal 1 : comparer les services (Wireshark sur `s3-eth3` et `s3-eth2`, filtre : `tcp.port==8080 && ip.src==10.0.0.4`)

```
mininet> h4 curl -I --max-time 3 http://10.0.0.3:8080
mininet> h1 curl -I --max-time 3 http://10.0.0.3:8080
mininet> h2 curl -I --max-time 3 http://10.0.0.3:8080
mininet> h4 ping -c 2 h3
```

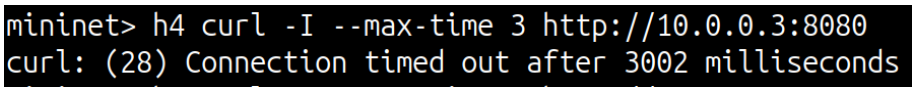


FIGURE 43 – Sortie de `h4 curl` avec l'ACL active

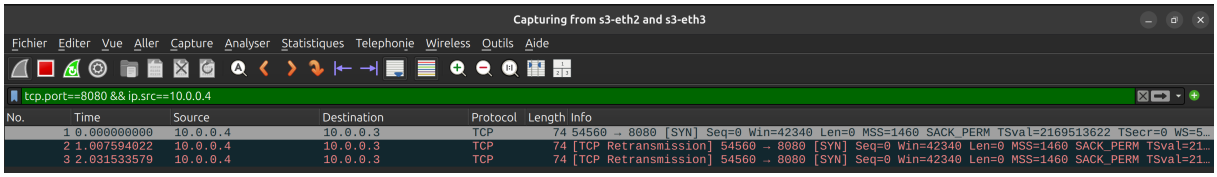


FIGURE 44 – Capture Wireshark : SYN de `h4` sur `s3-eth3` uniquement

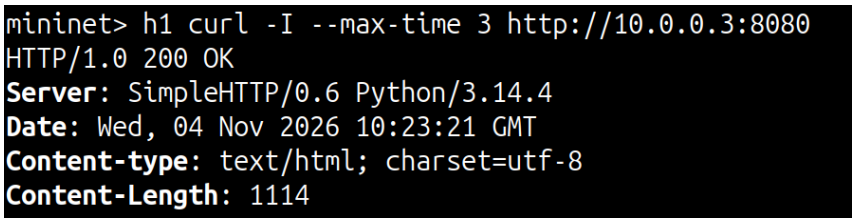


FIGURE 45 – Sortie de `h1 curl -I http://10.0.0.3:8080`

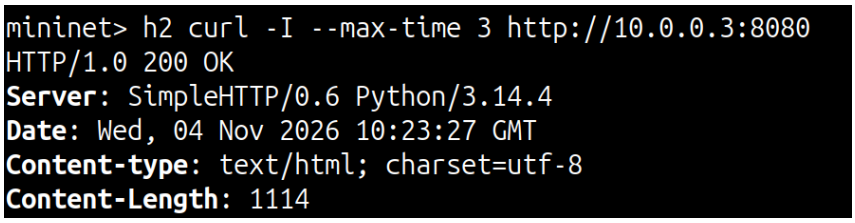


FIGURE 46 – Sortie de `h2 curl -I http://10.0.0.3:8080`

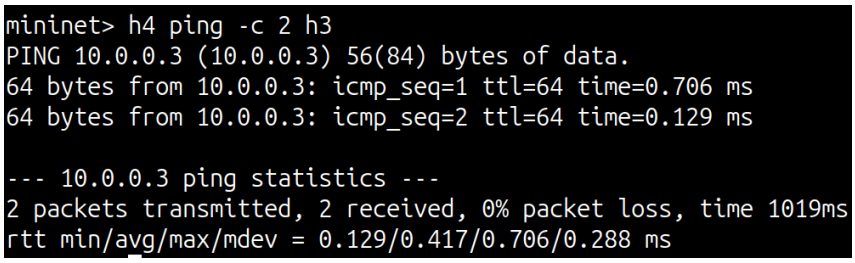


FIGURE 47 – Sortie de `h4 ping -c 2 h3`

Observation.

- `h4` → `h3` (HTTP) : `curl: (28) Connection timed out after 3002 milliseconds`. Wireshark montre le SYN de `h4` et ses deux retransmissions (`TCP Retransmission`) sur `s3-eth3`, sans SYN/ACK ;
- `h1` et `h2` → `h3` (HTTP) : `HTTP/1.0 200 OK` ;
- `h4` → `h3` (ping) : 2/2 reçus, RTT min/moyen/max = 0,129/0,417/0,706 ms.

L'ACL bloque seulement ce service depuis cet hôte : l'interdiction dépend à la fois de la source et du protocole TCP/8080.

6 Valider la politique et interpréter les résultats

Objectif : combiner trois types de preuves : le comportement des applications, les compteurs OVS et le trajet observé dans Wireshark.

Listing 19 – Terminal 2 : tables de flux et compteurs

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
sudo ovs-ofctl -O OpenFlow13 dump-flows s2
sudo ovs-ofctl -O OpenFlow13 dump-flows s3
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=813.723s, table=0, n_packets=3, n_bytes=294, priority=300,icmp,nw_src=10.0.0.2,nw_dst=10.0.0.3,icmp_type=8 actions=drop
 cookie=0x0, duration=1688.664s, table=0, n_packets=28, n_bytes=3516, priority=200,ip,nw_dst=10.0.0.1 actions=output:"s1-eth1"
 cookie=0x0, duration=1683.695s, table=0, n_packets=27, n_bytes=3602, priority=200,ip,nw_dst=10.0.0.2 actions=output:"s1-eth2"
 cookie=0x0, duration=1679.046s, table=0, n_packets=48, n_bytes=4064, priority=200,ip,nw_dst=10.0.0.3 actions=output:"s1-eth3"
 cookie=0x0, duration=1675.832s, table=0, n_packets=4, n_bytes=392, priority=200,ip,nw_dst=10.0.0.4 actions=output:"s1-eth3"
 cookie=0x0, duration=2249.965s, table=0, n_packets=76, n_bytes=3192, priority=100,arp actions=FLOOD
```

FIGURE 48 – `dump-flows s1` : compteur de l'ACL ICMP

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 dump-flows s2
 cookie=0x0, duration=1634.689s, table=0, n_packets=26, n_bytes=3320, priority=200,ip,nw_dst=10.0.0.1 actions=output:"s2-eth1"
 cookie=0x0, duration=1631.030s, table=0, n_packets=25, n_bytes=3406, priority=200,ip,nw_dst=10.0.0.2 actions=output:"s2-eth1"
 cookie=0x0, duration=1625.181s, table=0, n_packets=48, n_bytes=4064, priority=200,ip,nw_dst=10.0.0.3 actions=output:"s2-eth2"
 cookie=0x0, duration=1622.027s, table=0, n_packets=4, n_bytes=392, priority=200,ip,nw_dst=10.0.0.4 actions=output:"s2-eth2"
 cookie=0x0, duration=2275.412s, table=0, n_packets=76, n_bytes=3192, priority=100,arp actions=FLOOD
```

FIGURE 49 – `dump-flows s2` : switch de transit, sans ACL

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP6$ sudo ovs-ofctl -O OpenFlow13 dump-flows s3
 cookie=0x0, duration=402.807s, table=0, n_packets=3, n_bytes=222, priority=350,tcp,nw_src=10.0.0.4,nw_dst=10.0.0.3,tp_dst=8080 actions=drop
 cookie=0x0, duration=1590.107s, table=0, n_packets=26, n_bytes=3320, priority=200,ip,nw_dst=10.0.0.1 actions=output:"s3-eth1"
 cookie=0x0, duration=1586.500s, table=0, n_packets=25, n_bytes=3406, priority=200,ip,nw_dst=10.0.0.2 actions=output:"s3-eth1"
 cookie=0x0, duration=1582.007s, table=0, n_packets=57, n_bytes=4872, priority=200,ip,nw_dst=10.0.0.3 actions=output:"s3-eth2"
 cookie=0x0, duration=1577.071s, table=0, n_packets=13, n_bytes=1277, priority=200,ip,nw_dst=10.0.0.4 actions=output:"s3-eth3"
 cookie=0x0, duration=2298.471s, table=0, n_packets=76, n_bytes=3192, priority=100,arp actions=FLOOD
```

FIGURE 50 – `dump-flows s3` : compteur de l'ACL TCP

Observation. Compteurs relevés :

Switch	Priorité	Règle	n_packets	n_bytes
`s1`	300	ACL ICMP `h2` → `h3` (`icmp_type=8`) → `drop`	3	294
`s1`	200	`ip,nw_dst=10.0.0.3` → `s1-eth3`	48	4064
`s2`	200	`ip,nw_dst=10.0.0.3` → `s2-eth2`	48	4064
`s3`	350	ACL TCP `h4` → `h3` (`tp_dst=8080`) → `drop`	3	222
`s3`	200	`ip,nw_dst=10.0.0.3` → `s3-eth2`	57	4872

TABLE 5 – Compteurs de `dump-flows` (règles principales)

Les compteurs sont cumulatifs depuis l'installation des règles : ils ne correspondent pas au seul dernier test. L'ACL de `s1` affiche 3 paquets de 98 octets (294 octets), soit les trois Echo Request de `h2` ; l'ACL de `s3` affiche 3 paquets de 74 octets (222 octets), soit le SYN de `h4` et ses deux retransmissions. Le switch de transit `s2` ne contient aucune ACL : il se contente de relayer.

Test final	Attendu	Observation
`h1` → `h3` ping	OK	2/2 reçus, 0 % de perte
`h2` → `h3` ping	ÉCHEC	3 émis, 0 reçu, 100 % de perte ; Echo Request absents de `s1-eth3`
`h1` → `h3` HTTP 8080	OK	`HTTP/1.0 200 OK`
`h2` → `h3` HTTP 8080	OK	`HTTP/1.0 200 OK`
`h4` → `h3` HTTP 8080	ÉCHEC	`Connection timed out after 3002 ms` ; SYN absents de `s3-eth2`
`h4` → `h3` ping	OK	2/2 reçus, 0 % de perte

TABLE 6 – Validation de la matrice de sécurité

6.1 Questions de compréhension

1. **Pourquoi avoir installé ARP et le forwarding IPv4 avant les deux ACL ?** Une ACL n'a de sens que si la communication fonctionne avant son installation : sans ARP ni routage IPv4, tout échoue déjà et on ne pourrait pas attribuer l'échec à l'ACL. Le `pingall` à 12/12 et les `200 OK` de référence prouvent que seule l'ACL change ensuite le comportement.
2. **Pourquoi l'ACL ICMP est-elle sur `s1` et l'ACL TCP sur `s3` ?** Le ping de `h2` traverse `s1`, `s2` et `s3` ; on le bloque sur `s1`, au plus près de la source, pour ne pas le laisser circuler dans le réseau. Le trafic `h4` → `h3` entre par `s3-eth3` et sort par `s3-eth2` sans jamais passer par `s1` ni `s2` : `s3` est le seul switch où l'ACL peut agir.
3. **En quoi `priority=300` ou `350` modifie-t-elle le comportement des règles IPv4 de priorité 200 ?** Un paquet bloqué correspond à la fois à la règle IPv4 (200) et à l'ACL (300 ou 350) ; c'est la priorité la plus élevée qui s'applique, donc l'action `drop` prend le pas sur `output:N`. Les paquets qui ne correspondent qu'à la règle IPv4 continuent d'être transmis.
4. **Que prouvent conjointement l'absence du paquet sur le port de sortie et l'augmentation de `n_packets` ?** Que le paquet est bien arrivé au switch, a été reconnu par la règle de blocage puis abandonné : il n'a pas été perdu sur le lien, c'est la politique de transfert qui l'a supprimé.

7 Clôture du TP

Ordre obligatoire : arrêter et enregistrer les captures Wireshark **avant** de nettoyer Mininet, sinon les interfaces de capture disparaissent.

Listing 20 – Fermeture de Mininet puis nettoyage

```
mininet> exit
sudo mn -c
```

Observation. `exit` arrête le réseau (hôtes, switches et liens) ; `sudo mn -c` nettoie les interfaces virtuelles et les switches temporaires.

8 Synthèse

Étape	Politique	Règles	Résultat observé
2	Tables vides	Aucune	Ping en échec, `Host Unreachable`
3.1	ARP seul	`priority=100,arp,FLOOD` (`s1`, `s2`, `s3`)	ARP résolu, ping 100 % de perte
3.2	Forwarding IPv4	12 règles `priority=200`	`pingall` 12/12, HTTP `200 OK` pour `h1`, `h2`, `h4`
4	ACL ICMP sur `s1`	`priority=300`, `icmp_type=8`, `h2` → `h3`	Ping de `h2` bloqué, HTTP et ping de `h1` OK
5	ACL TCP sur `s3`	`priority=350`, `tcp_dst=8080`, `h4` → `h3`	HTTP de `h4` en timeout, ping de `h4` OK

TABLE 7 – Récapitulatif des étapes

Réponse à la question de départ. Oui : en combinant source, destination, protocole, port et priorité, une ACL interdit un protocole à un utilisateur sans toucher aux autres services. Deux politiques différentes ont été appliquées sans changer l'architecture : l'ACL ICMP sur `s1` bloque le ping initié par `h2`, l'ACL TCP sur `s3` bloque le service HTTP depuis `h4`, chacune sur le switch par lequel le trafic visé passe réellement.

Conclusion. À la fin du TP, je sais lire une topologie multi-switches et ses ports OpenFlow, programmer ARP et le forwarding IPv4 sur plusieurs switches, installer des ACL ciblées (ICMP par type, TCP par port) avec la bonne priorité sur le bon switch, et prouver leur effet avec les messages de `ping` et `curl`, les compteurs de `dump-flows` et Wireshark.